DA3408 Assignment 3: Spark vs. Ray

Ramana Gopal Kodali
DA24B010

October 5, 2026

Abstract

This experiment implements an equivalent NYC Yellow Taxi preprocessing pipeline in Apache Spark and Ray Data and compares their measured behavior on 36 monthly inputs. The pipeline performs ingestion, cleansing, timestamp formatting, duplicate removal, taxi-zone enrichment, average-speed computation, and Parquet export. Both native and Python-UDF variants were executed. Spark completed the native pipeline in 392.50 s and the UDF pipeline in 432.34 s, while Ray required 636.96 s and 711.03 s respectively. Thus Spark was the end-to-end winner in this tested configuration. The Python UDF increased runtime by 10.15% in Spark and 11.63% in Ray. All four runs reached a 100% instantaneous CPU peak, while Ray showed lower observed peak physical RAM. Native Spark/Ray outputs matched for all 36 monthly files, totaling 127,627,157 rows each.

1 1. Objective and Assignment Requirements

The assignment requires an identical preprocessing pipeline in Apache Spark and Ray, deployed as distributed two-worker clusters and benchmarked on the NYC Taxi dataset. The required pipeline consists of: (1) ingestion of multiple files, (2) cleansing including null filtering, duplicate removal and timestamp formatting, (3) a heavy join with location information plus a derived feature using a custom Python function/UDF, and (4) Parquet export for ML use. The required measurements are total ingestion-to-export wall-clock time, peak CPU and memory, and Python-UDF overhead. [**?**]

The assignment also requires exact pipeline parity, cluster/UI evidence, a UDF deep-dive, a winner conclusion, and a brief AI-first versus BI-first discussion. Benchmark values must come from actual executions rather than synthetic or AI-generated numbers.

2 2. System, Cluster and Common Pipeline

Environment. Ubuntu 24.04.4 LTS ARM64 VM; Python 3.10.13; Java 17; Apache Spark 4.2.0; Ray 2.20.0; PyArrow 15.0.2. Spark used a master plus two workers. Ray used a head plus two worker processes. Cluster and network configuration were performed manually, as required by the assignment.

Common logical pipeline.

1. **Ingestion:** read the 36 monthly Yellow Taxi Parquet files.
2. **Cleansing:** normalize types, parse pickup/dropoff timestamps, remove null/invalid records, and remove duplicates.
3. **Heavy join:** join trip `PULocationID` with `taxi_zone_lookup.csv`.
4. **Derived feature:** compute

$$\texttt{average_speed_mph} = \frac{\texttt{trip_distance} \times 3600}{\texttt{duration_seconds}}.$$

5. **Export:** write the structured result to monthly Parquet output.

The native mode computes average speed with framework-native expressions. The UDF mode replaces that expression with a custom Python function. The logical output schema and cleansing semantics were kept aligned for parity.

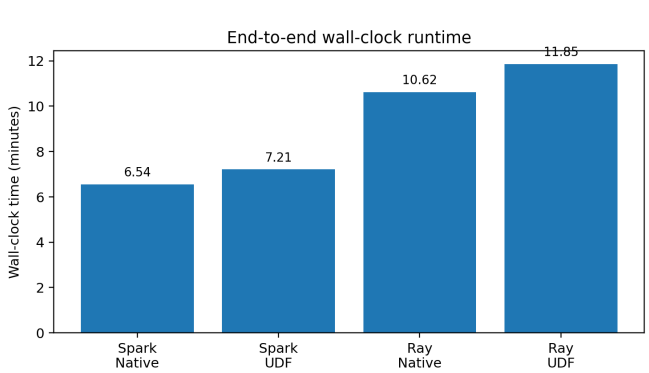
3 3. Benchmark Method and Results

A single end-to-end timer covered ingestion through export for each complete 36-month run. The monthly processing strategy was used to control memory, but the reported values are four complete benchmarks, not 144 independent experiments. CPU and RAM were collected from VM-level resource logs. Peak CPU is the maximum sampled utilization; peak RAM is the maximum sampled used physical memory.

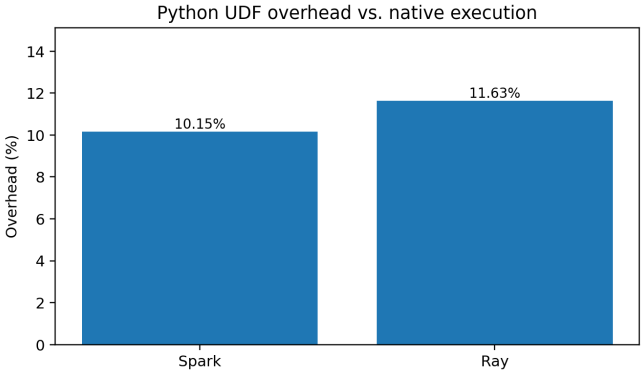
Finding 1 — Execution time. Spark is the clear winner for this measured workload. Spark Native is 38.38% faster than Ray Native, and Spark UDF is 39.20% faster than Ray UDF. The result is specific to the tested pipeline, dataset, cluster configuration and VM; it is not a universal ranking of the frameworks.

Table 1: Recorded end-to-end benchmark results.

Framework	Mode	Time (s)	Time (min)	Relative to Spark Native
Spark	Native	392.50	6.54	1.00×
Spark	UDF	432.34	7.21	1.10×
Ray	Native	636.96	10.62	1.62×
Ray	UDF	711.03	11.85	1.81×



(a) End-to-end runtime



(b) Python-UDF overhead

4 4. Python UDF Deep-Dive

The required overhead is

$$\text{UDF overhead} = \frac{T_{\text{UDF}} - T_{\text{Native}}}{T_{\text{Native}}} \times 100.$$

For Spark,

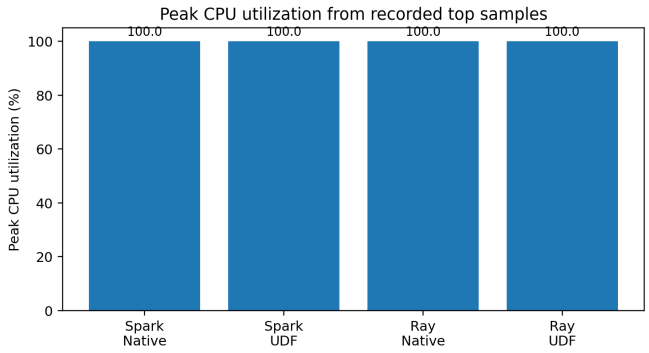
$$\frac{432.3402 - 392.5020}{392.5020} \times 100 = 10.15\%.$$

For Ray,

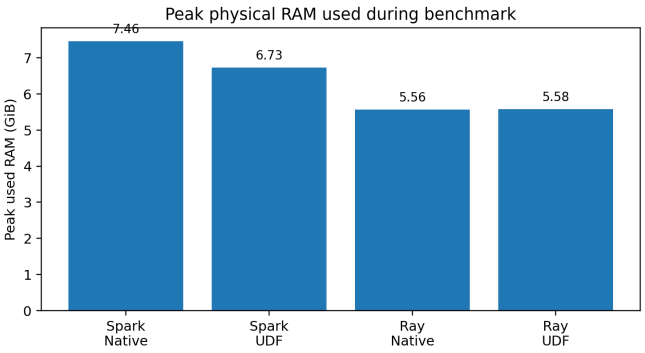
$$\frac{711.0328 - 636.9566}{636.9566} \times 100 = 11.63\%.$$

Therefore, introducing the Python transformation increased the complete pipeline by 39.84 s in Spark and 74.08 s in Ray. Ray's measured UDF overhead was 1.48 percentage points higher. The experiment does **not** demonstrate a Python-native speed advantage for Ray on this workload. The correct interpretation is empirical: both frameworks slowed when the native expression was replaced by Python-level computation, and Ray slowed more in this end-to-end test.

5 5. Resource Utilization



(a) Peak CPU utilization



(b) Peak physical RAM used

Finding 2 — Resource behavior. All four runs reached the same instantaneous 100% CPU peak, so peak CPU alone does not distinguish the frameworks. Ray had the lower observed peak RAM, around 5.6 GiB, compared with 6.7–7.5 GiB for Spark. However, higher average CPU utilization did not translate into lower elapsed time: Spark completed the workload substantially faster. The result therefore shows a trade-off between memory footprint and wall-clock throughput rather than a single framework winning every resource metric.

Table 2: Resource measurements from the recorded VM logs.

Run	Peak CPU	Mean CPU	P95 CPU	Peak RAM
Spark Native	100.0%	65.9%	95.7%	7.46 GiB
Spark UDF	100.0%	68.3%	100.0%	6.73 GiB
Ray Native	100.0%	76.6%	98.3%	5.56 GiB
Ray UDF	100.0%	72.8%	97.9%	5.58 GiB

6 6. Pipeline Parity and Correctness

Correctness was checked before interpreting performance. The final native comparison matched Spark and Ray for **all 36 monthly outputs**, with 127,627,157 rows on each side. A detailed January comparison found 3,065,645 rows on each side, zero duplicate rows, zero rows present only in one result, and zero checked formula/value mismatches.

The parity process required the implementations to agree on timestamp precision, duration calculation, nullable integer handling, taxi-zone CSV null/string semantics, duplicate-removal behavior, source columns, and the average-speed formula. These details are important because superficially similar Spark and Ray programs can otherwise produce different logical results.

For the UDF implementation, the rigorous logical-row verification passed for 18 consecutive monthly outputs before the exhaustive check was stopped to avoid another long verification pass. Therefore, this report claims full 36-month exact parity for the native comparison, while the UDF statement is limited to the 18-month verification actually completed.

7 7. Performance Tuning Note

An early Ray implementation attempted global all-to-all duplicate removal. That approach caused severe memory pressure and an out-of-memory failure. The final implementation preserved the intended semantics while changing the execution strategy: exact month-level deduplication, reading only required source columns, bounded map-batch sizes, multiple Ray blocks, an object-store memory cap, and controlled output block sizes.

This tuning was necessary for a fair *completed* benchmark: a theoretically direct implementation that cannot finish under the available VM memory is not a useful production comparison. The final Ray implementation completed the full 36-month workload and passed the native parity checks.

8 8. Monthly Behavior

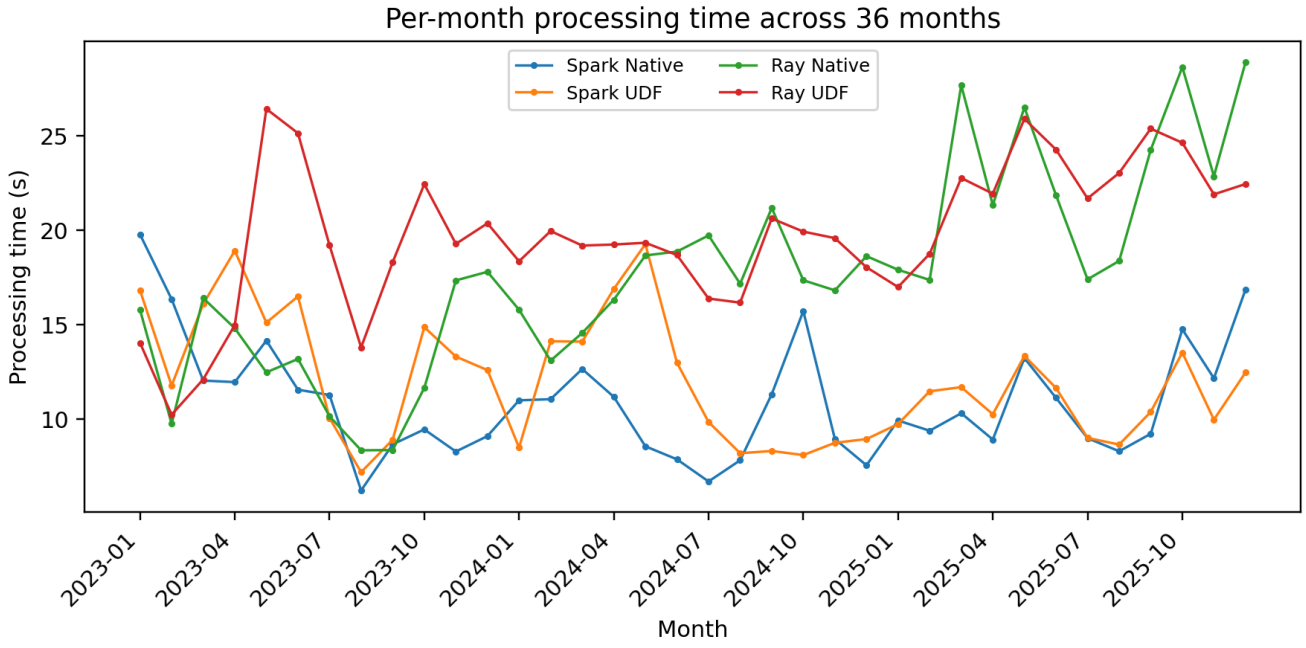


Figure 3: Recorded processing time for each of the 36 monthly inputs. The monthly timings show workload variation rather than assuming a constant cost per month.

The monthly measurements were retained as part of the benchmark evidence. The final comparison is based on the accumulated ingestion-to-export time across the same 36 inputs, rather than on a synthetic or estimated workload.

9 9. Findings Against the Assignment Rubric

Table 3: Summary of findings aligned with the assignment requirements.

Requirement	Finding
Cluster orchestration	Spark: master + two workers. Ray: head + two workers. UI/resource evidence was collected separately.
Pipeline parity	Same ingestion, cleansing, zone join, derived feature and Parquet export logic. Native outputs matched across all 36 months.
Execution time	Spark Native 392.50 s vs Ray Native 636.96 s; Spark UDF 432.34 s vs Ray UDF 711.03 s.
Resource spikes	All runs reached 100% peak CPU. Ray had lower peak RAM (5.56–5.58 GiB).
UDF deep-dive	Spark overhead 10.15%; Ray overhead 11.63%. No measured Python-native speed advantage for Ray here.
Winner	**Spark for this experiment**, based on substantially lower end-to-end wall-clock time.
BI-first vs AI-first	**BI-first/structured ETL: Spark** based on this benchmark. **AI-first/flexible Python workloads: Ray remains architecturally attractive**, but this experiment did not demonstrate a speed advantage for it.

10 10. Conclusion

For the exact workload and VM configuration tested, **Spark is the benchmark winner**. It completed the native pipeline in 392.50 s versus 636.96 s for Ray and the UDF pipeline in 432.34 s versus 711.03 s. Spark's UDF overhead was 10.15%, compared with 11.63% for Ray. Ray used less peak physical RAM, but both frameworks reached a 100% instantaneous CPU peak and Spark delivered substantially lower wall-clock time.

The key engineering conclusion is therefore nuanced: Spark is the better choice for this measured structured ETL workload, while Ray remains a reasonable AI-first platform when flexible Python-centric execution and ML-oriented task parallelism are primary design requirements. The benchmark does not justify claiming that Ray is faster for this dataset.

11. AI Usage Disclosure

Tools used: ChatGPT and Claude.

How used: AI tools were used as learning and development assistants for translating equivalent PySpark/Ray logic, debugging Ray memory issues, explaining Spark/Ray concepts, constructing parity and benchmark-analysis scripts, and organizing documentation.

Impact: AI assistance helped adapt and debug the implementations, improve memory management, verify parity, and organize the benchmark analysis. Cluster and network configuration were performed manually. No synthetic or fabricated benchmark values were used; all reported measurements came from actual executions and recorded logs. The student reviewed and executed the final code and remains responsible for understanding and defending the submission.